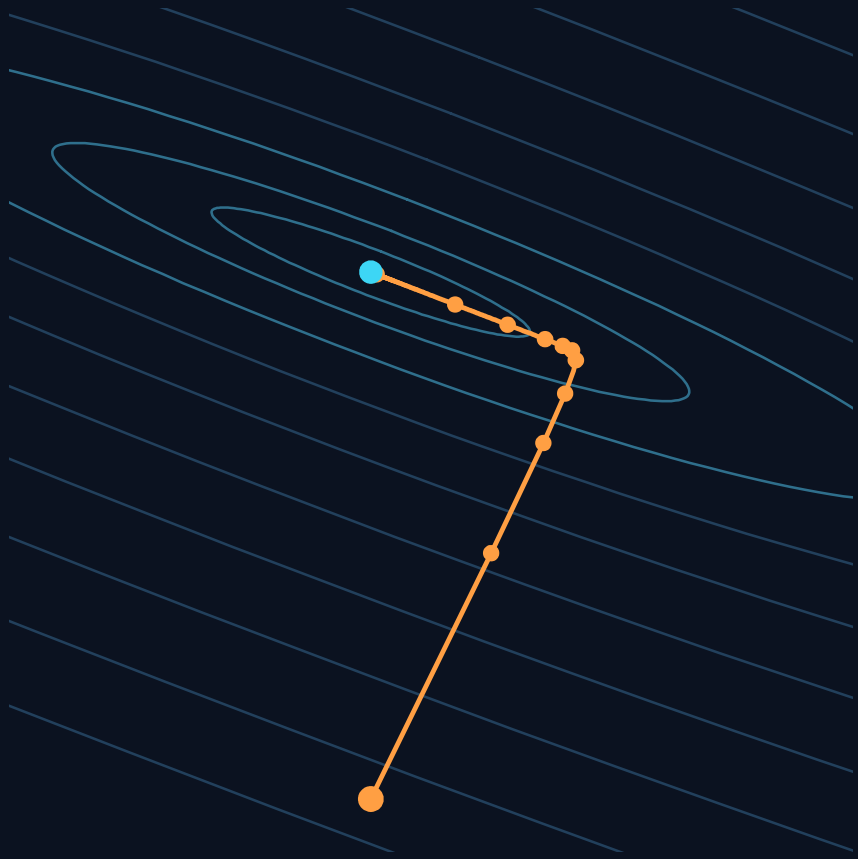
MÓDULO 06 · MACHINE LEARNING NA MÃO, COM NUMPY

Gradiente descendente do zero

A mesma regressão linear da aula 6.2, agora encontrada passo a passo: função de custo, gradiente, taxa de aprendizado, convergência e a descida animada sobre a superfície de custo.

AULA 6.3 2 HORAS PRÁTICA: ANIMAR A DESCIDA



Objetivos e roteiro da aula

Ao final da aula, você vai conseguir:

01 Explicar o significado de β_0, β_1, α, $J(\beta)$ e ∇J, e onde cada um aparece no código.

02 Calcular à mão uma iteração completa do gradiente descendente.

03 Implementar o gradiente descendente vetorizado em NumPy, sem otimizador pronto.

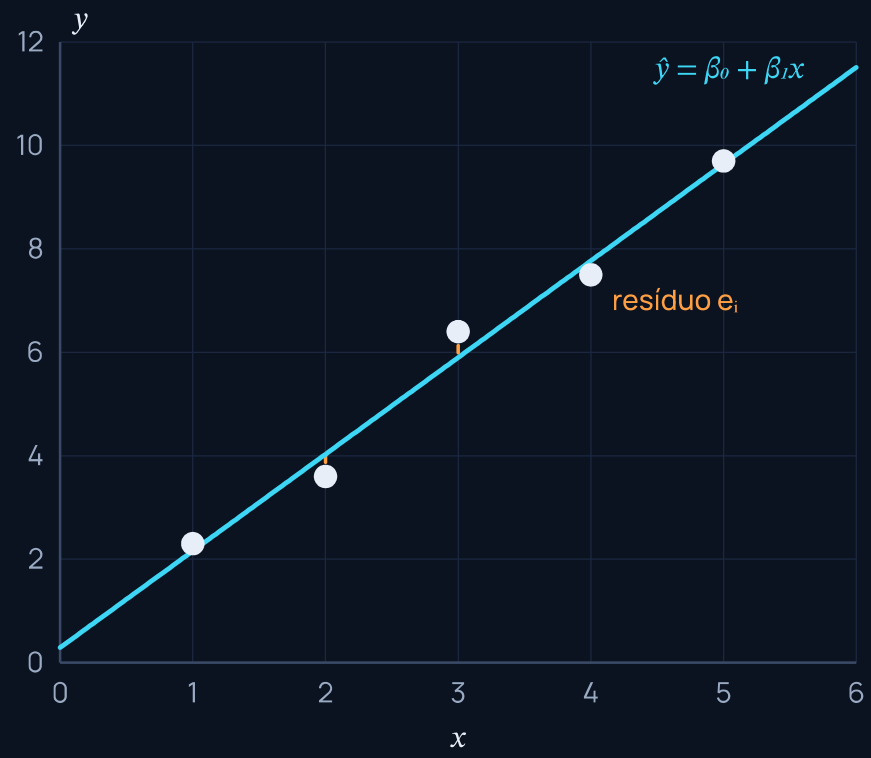
04 Diagnosticar convergência e divergência pela curva de custo.

05 Animar a trajetória dos coeficientes sobre as curvas de nível do custo.

ROTEIRO · 120 MINUTOS

00–10	Objetivos e recapitulação
10–35	Teoria: custo e gradiente
35–40	Quiz rápido
40–55	Primeira iteração à mão
55–70	α, épocas e variantes
70–85	NumPy vetorizado
85–110	Exercício guiado no notebook
110–120	Erros, desafio e leitura

Na aula 6.2, β saiu de uma única conta

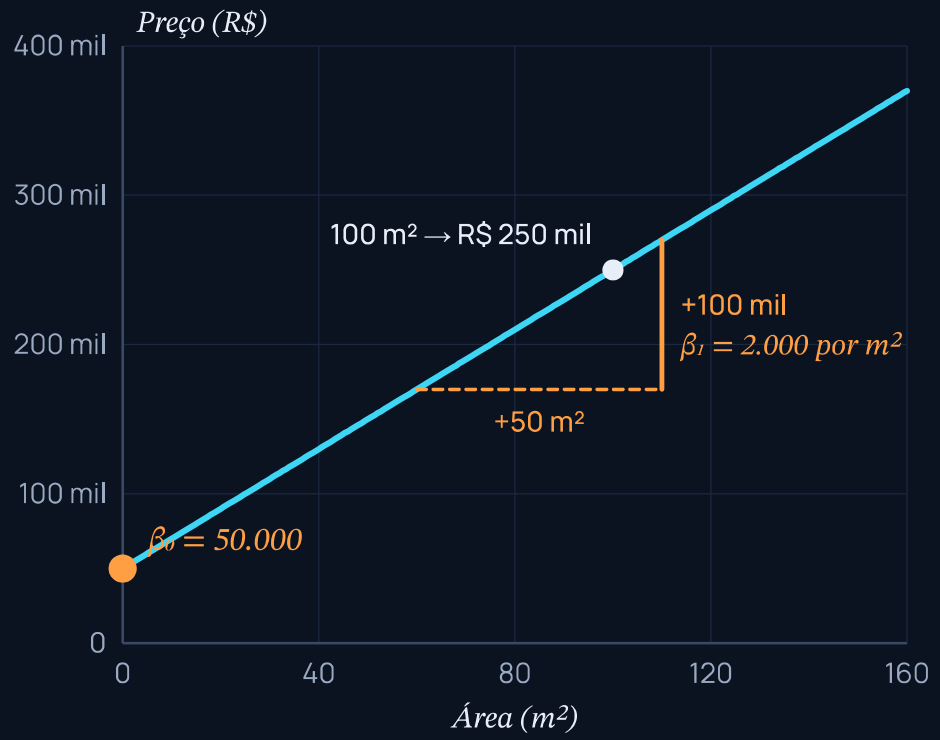


MODELO	$\hat{y} = \beta_0 + \beta_1 x$
RESÍDUO	$e_i = y_i - \hat{y}_i$
MÍNIMOS QUADRADOS	minimizar $\Sigma\, e_i^2$
EQUAÇÃO NORMAL	$\beta = (X^T X)^{-1} X^T y$
PSEUDOINVERSA	$\beta = X^{+} y$ `np.linalg.pinv(X) @ y`

LEGENDA e_i resíduo da amostra i · Σ soma de $i = 1$ até m · X^T transposta de X · $(\cdot)^{-1}$ inversa · X^{+} pseudoinversa

Cada coeficiente tem um significado concreto

Exemplo: preço de um imóvel previsto pela área.



$$\text{Preço} = 50.000 + 2.000 \times \text{Área}$$

$\hat{y}$ **y previsto**
O valor que o modelo calcula: o preço.

x **entrada**
A variável usada para prever: a área.

β_0 **intercepto**
Valor de ŷ quando x = 0. Aqui, 50.000.

β_1 **coeficiente angular**
Quanto ŷ muda por unidade de x. Aqui, +2.000 por m².

β **parâmetros**
Os números que o modelo precisa aprender.

Dois métodos para encontrar os mesmos coeficientes

AULA 6.2

Equação normal

$$\beta = (X^{\mathrm{T}}X)^{-1}X^{\mathrm{T}}y$$

- Calcula a solução de mínimos quadrados diretamente, com álgebra linear.
- Uma única conta; nenhum parâmetro de treino.
- Fica cara quando há muitas características (inversão cresce com n^3).

AULA 6.3

Gradiente descendente

$$\beta \leftarrow \beta - \alpha \ \nabla J(\beta)$$

- Começa com coeficientes iniciais e os modifica repetidamente para diminuir o erro.
- Exige escolher a taxa de aprendizado e o número de épocas.
- Escala para grandes volumes de dados e é a base do treino de redes neurais.

LEGENDA Mesmo modelo e mesma função de erro; muda só o método. ← recebe: o valor da esquerda é substituído · α taxa de aprendizado, o tamanho do passo · ∇J gradiente, a direção em que o erro sobe

Descer uma montanha no escuro

Uma pessoa com neblina densa não enxerga o vale. Ela só sente a inclinação do chão onde está pisando.

01	**Altura**	Quão grande está o erro na posição atual.
02	**Inclinação sob os pés**	Para que lado o terreno sobe e com que força.
03	**Andar contra a subida**	Dar o passo no sentido em que o terreno desce.
04	**Tamanho do passo**	Passos curtos chegam devagar; passos longos podem pular o vale.
05	**Chão quase plano**	A inclinação ficou próxima de zero: estamos perto do fundo.

A montanha é a função de custo

Dados x = [1, 2, 3], y = [2, 4, 6], com β_0 fixo em 0. A bola mostra o gradiente descendente com $\alpha = 0{,}05$.



Altura — $J(\beta)$

valor da função de custo

Posição — β

valores atuais dos coeficientes

Inclinação — $\partial J/\partial\beta$

derivada do custo

Passo — $-\alpha \cdot \partial J/\partial\beta$

atualização dos coeficientes

1° PASSO Em $\beta_1 = 0$ a inclinação vale −9,33. Passo: $\beta_1 \leftarrow 0 - 0{,}05 \times (-9{,}33) = 0{,}467$. A fórmula de J vem no próximo slide.

A função de custo mede o erro do modelo

$$J(\beta) = \frac{1}{2m} \Sigma \left(y_i - \hat{y}_i \right)^2$$

J	custo: um único número que resume o erro	β	conjunto de parâmetros (β_0, β_1)	m	quantidade de amostras
i	índice da amostra: 1, 2, ..., m	y_i	valor real da amostra i	$\hat{y}_i$	previsão para a amostra i: $\hat{y}_i = \beta_0 + \beta_1 x_i$
Σ	somatório: soma o termo para todo i	$(y_i - \hat{y}_i)^2$	erro da amostra i ao quadrado	$1/(2m)$	média, dividida por 2 para simplificar a derivada

NO CÓDIGO `y_hat = b0 + b1 * x` `custo = ((y - y_hat) ** 2).sum() / (2 * m)`

Erro, SSE, MSE e J medem a mesma coisa em escalas diferentes

			ERROS 1, 2, 3
Erro individual	$e_i = y_i - \hat{y}_i$	um número por amostra; pode ser negativo	1, 2, 3
Soma dos erros quadráticos	$\text{SSE} = \Sigma\, e_i^2$	cresce com a quantidade de dados	1 + 4 + 9 = 14
Erro quadrático médio	$\text{MSE} = \frac{1}{m}\, \Sigma\, e_i^2$	média: não depende de m	14 ÷ 3 = 4,67
Função de custo desta aula	$J = \frac{1}{2m}\, \Sigma\, e_i^2 = \frac{\text{MSE}}{2}$	metade do MSE	14 ÷ 6 = 2,33

CONSEQUÊNCIA J e MSE diferem por um fator positivo constante. O mesmo β minimiza os dois.

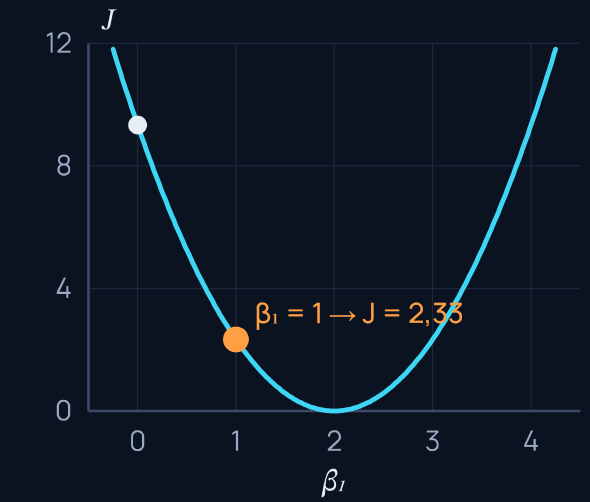
Calculando o custo à mão

Palpite: $\beta_0 = 0$ e $\beta_1 = 1$, ou seja, $\hat{y} = x$.

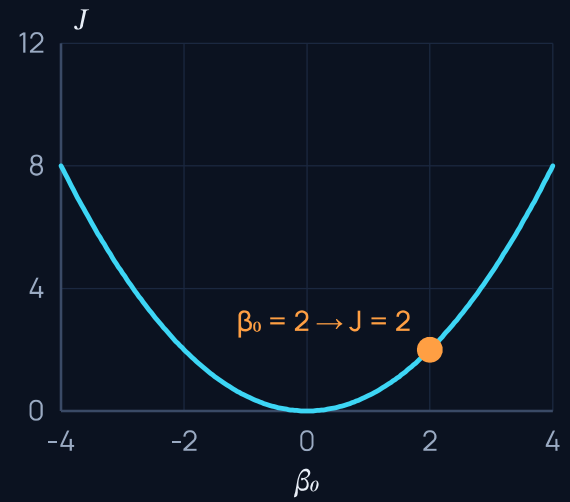
i	x_i	y_i	$\hat{y}_i$	$y_i - \hat{y}_i$	$(\ldots)^2$
1	1	2	1	1	1
2	2	4	2	2	4
3	3	6	3	3	9

$$J = \frac{1 + 4 + 9}{2 \cdot 3} = \frac{14}{6} = 2{,}33$$

VARIANDO B_1 (B_0 = 0)

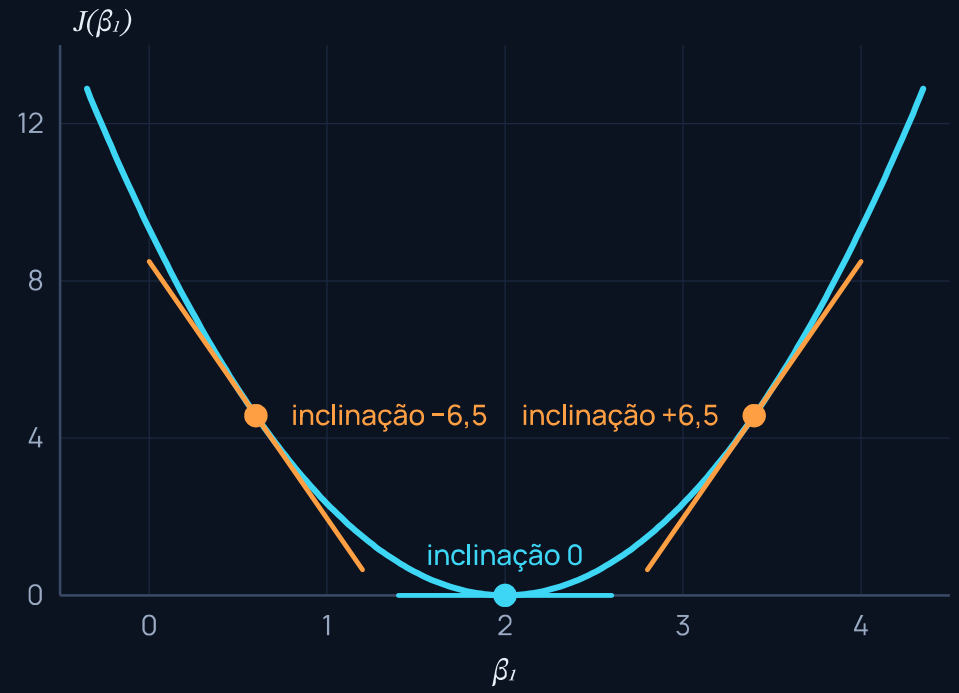


VARIANDO B_0 (B_1 = 2)



PERGUNTA Calcule J para $\beta_0 = 0$ e $\beta_1 = 3$. O resultado é maior ou menor que 2,33?

A derivada indica para onde o custo sobe



DEFINIÇÃO PRÁTICA

$$\frac{dJ}{d\beta} \approx \frac{J(\beta + h) - J(\beta)}{h}$$

Em $\beta_1 = 0{,}6$ e $h = 0{,}01$: $(4{,}5082 - 4{,}5733) \div 0{,}01 \approx -6{,}5$

Derivada positiva

custo sobe quando β aumenta → para descer: diminuir β

Derivada negativa

custo desce quando β aumenta → para descer: aumentar β

Derivada ≈ 0

curva quase plana → para descer: β quase não muda: perto do mínimo

PERGUNTA Se a derivada vale −0,001, o modelo está longe ou perto do mínimo?

Uma derivada parcial para cada coeficiente

DERIVADA PARCIAL · ∂

$$\frac{\partial J}{\partial \beta_0} \quad \text{e} \quad \frac{\partial J}{\partial \beta_1}$$

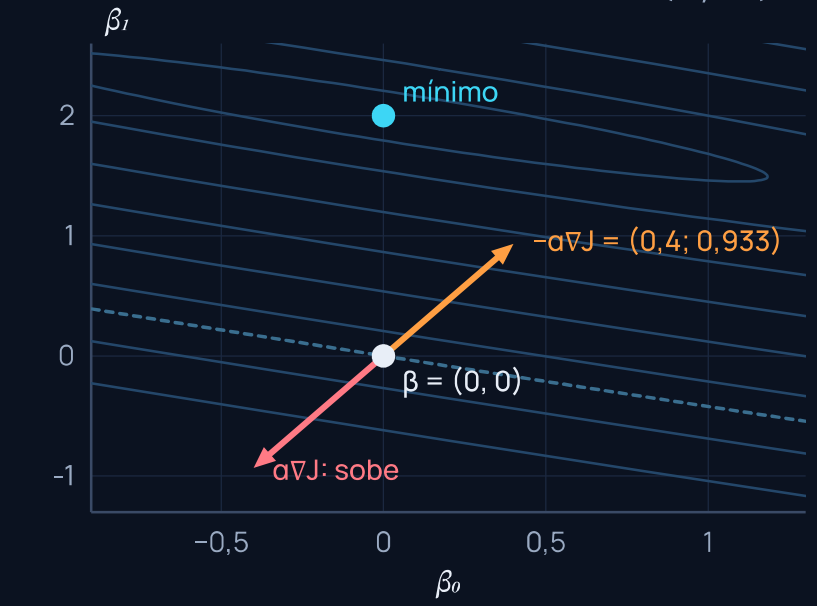
Mexe em um coeficiente e mantém o outro congelado. Mede quanto o custo muda nessa direção.

GRADIENTE · ∇

$$\nabla J(\beta) = \begin{bmatrix} \frac{\partial J}{\partial \beta_0} \\ \frac{\partial J}{\partial \beta_1} \end{bmatrix} \qquad \nabla J(0,0) = \begin{bmatrix} -4 \\ -9{,}33 \end{bmatrix}$$

Vetor com todas as derivadas parciais. Aponta para onde o custo sobe mais rápido; o algoritmo anda no sentido oposto.

MAPA DE J COM O GRADIENTE EM B = (0, 0)



A regra de atualização

$$\beta_j \leftarrow \beta_j - \alpha \frac{\partial J}{\partial \beta_j}$$

- β_j (esquerda): valor atualizado
- $\leftarrow$: recebe
- β_j (direita): valor anterior
- α: tamanho do passo, $\alpha > 0$
- $\frac{\partial J}{\partial \beta_j}$: inclinação: direção de ajuste

Derivada	Passo $-\alpha \cdot$ derivada	Efeito em β_j
positiva	negativo	diminui
negativa	positivo	aumenta
≈ 0	≈ 0	quase não muda

EXEMPLO

$$\beta_1 = 0 - 0{,}1 \times (-9{,}33) = 0{,}933$$

Derivada negativa → β_1 aumenta.

NO CÓDIGO j é o índice do coeficiente: $j = 0$ para β_0, $j = 1$ para β_1. Todos de uma vez: `beta = beta - alpha * gradiente`

De onde vêm as derivadas

1 Custo, com o erro escrito como $\hat{y}_i - y_i$

$$J = \frac{1}{2m} \Sigma (\hat{y}_i - y_i)^2, \quad \hat{y}_i = \beta_0 + \beta_1 x_i$$

$(a - b)^2 = (b - a)^2$: o custo não muda.

2 Regra da cadeia: derivada de fora × derivada de dentro

$$\frac{\partial J}{\partial \beta_j} = \frac{1}{2m} \Sigma \, 2(\hat{y}_i - y_i) \cdot \frac{\partial \hat{y}_i}{\partial \beta_j}$$

O 2 do quadrado cancela o 1/2.

3 Derivada da previsão

$$\frac{\partial \hat{y}_i}{\partial \beta_0} = 1 \qquad \frac{\partial \hat{y}_i}{\partial \beta_1} = x_i$$

β_0 aparece sozinho; β_1 multiplica x_i.

PARA B_0

$$\frac{\partial J}{\partial \beta_0} = \frac{1}{m} \Sigma (\hat{y}_i - y_i)$$

Erro médio.

PARA B_1

$$\frac{\partial J}{\partial \beta_1} = \frac{1}{m} \Sigma (\hat{y}_i - y_i) \, x_i$$

Erro médio ponderado pela entrada x_i.

NO CÓDIGO

```
erro = y_hat - y   grad_b0 = erro.sum() / m   grad_b1 = (erro * x).sum() / m
```

Quiz rápido

Responda antes de seguir. A última pergunta é no caderno.

01 Em uma iteração, $\partial J/\partial \beta_1 = +3$. β_1 vai aumentar ou diminuir?

02 No modelo Preço = 50.000 + 2.000 × Área, o que β_0 representa?

03 Se $\nabla J \approx (0, 0)$, o que está acontecendo com o treinamento?

04 No caderno: $\beta = (0, 1)$, $\alpha = 0{,}1$, dados $x = [1, 2, 3]$, $y = [2, 4, 6]$. Calcule o novo β_1.

Primeira iteração: previsões, erros e custo

$x = [1, 2, 3] \cdot y = [2, 4, 6] \cdot \beta_0 = 0 \cdot \beta_1 = 0 \cdot \alpha = 0{,}1 \cdot m = 3$

ETAPAS 1 E 2 · PREVISÕES E ERROS

i	x_i	y_i	$\hat{y}_i = 0 + 0 \cdot x_i$	$\hat{y}_i - y_i$	$(\ldots)^2$
1	1	2	0	−2	4
2	2	4	0	−4	16
3	3	6	0	−6	36

ETAPA 3 · CUSTO INICIAL

$$J(0,0) = \frac{(-2)^2 + (-4)^2 + (-6)^2}{2 \cdot 3}$$

$$= \frac{4 + 16 + 36}{6} = \frac{56}{6} = 9{,}3333$$

Todos os erros são negativos: a reta inicial está abaixo de todos os pontos.

Primeira iteração: gradientes e atualização

ETAPA 4 · GRADIENTES

$$\frac{\partial J}{\partial \beta_0} = \frac{(-2) + (-4) + (-6)}{3} = \frac{-12}{3} = -4$$

$$\frac{\partial J}{\partial \beta_1} = \frac{(-2)\cdot 1 + (-4)\cdot 2 + (-6)\cdot 3}{3}$$

$$= \frac{-2 - 8 - 18}{3} = \frac{-28}{3} = -9{,}3333$$

ETAPA 5 · ATUALIZAÇÃO COM A = 0,1

$$\beta_0 \leftarrow 0 - 0{,}1 \times (-4) = 0{,}4$$

$$\beta_1 \leftarrow 0 - 0{,}1 \times (-9{,}3333) = 0{,}9333$$

Os dois gradientes são negativos, então os dois coeficientes aumentam. β_1 aumenta mais porque cada erro foi multiplicado pelo seu x_i.

PERGUNTA Por que β_1 andou mais que β_0 nesta iteração?

Primeira iteração: o custo caiu de 9,33 para 1,88

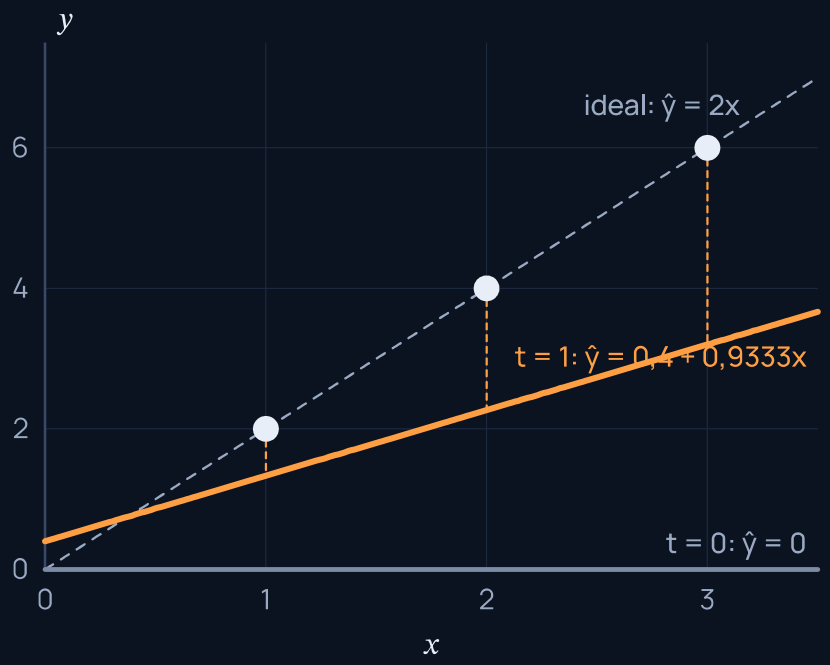
t: índice da iteração. $t = 0$ são os valores iniciais; $t = 1$, os valores após a primeira atualização.

ETAPA 6 · NOVAS PREVISÕES · ETAPA 7 · NOVO CUSTO

x_i	$\hat{y}_i = 0{,}4 + 0{,}9333x_i$	$y_i - \hat{y}_i$	$(\ldots)^2$
1	1,3333	0,6667	0,4444
2	2,2667	1,7333	3,0044
3	3,2	2,8	7,84

$$J = \frac{11{,}2889}{6} = 1{,}8815$$

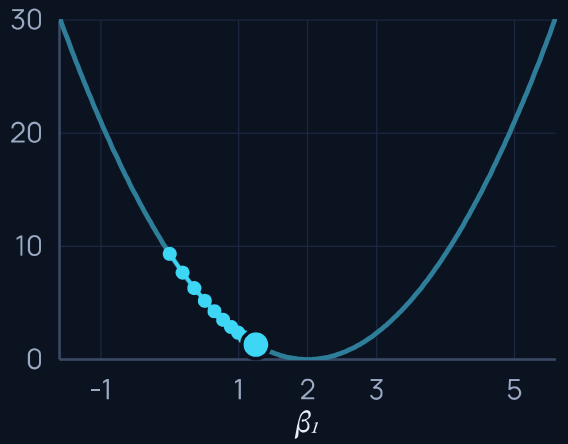
	Antes (t = 0)	Depois (t = 1)
β_0	0	0,4
β_1	0	0,9333
J	9,3333	1,8815



A taxa de aprendizado controla o tamanho do passo

Mesma função de custo, mesmo ponto inicial ($\beta_1 = 0$), 10 passos para cada α.

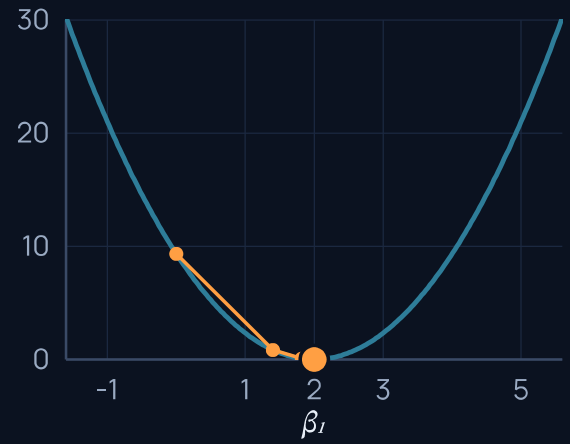
MUITO PEQUENA



Melhora lentamente.

`α = 0,02 · J após 10 passos: 1,315`

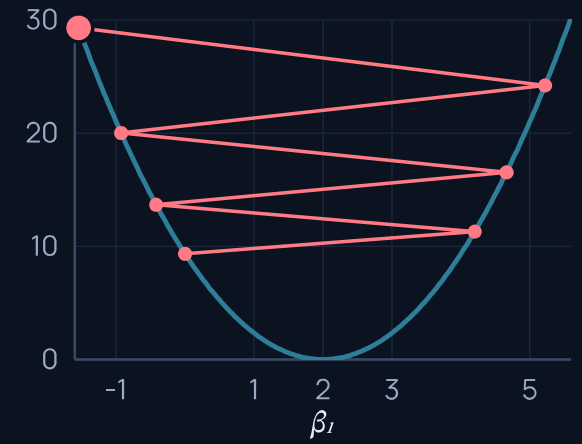
ADEQUADA



Desce de forma estável.

`α = 0,15 · J após 10 passos: ≈ 0`

MUITO ALTA



Ultrapassa o mínimo e diverge.

`α = 0,45 · J após 10 passos: 62,79`

POR QUÊ Neste corte, $\beta_1 \leftarrow \beta_1 - \alpha \cdot 4{,}667(\beta_1 - 2)$: cada passo multiplica a distância até o mínimo por $(1 - 4{,}667\alpha) = 0{,}907 \cdot 0{,}30 \cdot -1{,}10$. Não existe α universal: o valor seguro depende da curvatura do custo, que depende da escala dos dados.

Épocas, iterações e convergência

Iteração
Uma atualização dos coeficientes.

Época
Uma passagem completa por todas as m amostras.

Convergência
O custo para de cair de forma relevante.

Divergência
O custo cresce, oscila ou vira inf / nan.

CRITÉRIO DE PARADA

$$|J(t-1) - J(t)| < \varepsilon$$

CONVERGÊNCIA · A = 0,1



DIVERGÊNCIA · A = 0,4



EXEMPLO m = 1.000 amostras e 50 épocas → batch: 50 iterações · mini-batch (b = 100): 500 · SGD: 50.000. ε: tolerância, ex.: 10^{-8}. t: índice da iteração.

Batch, estocástico e mini-batch

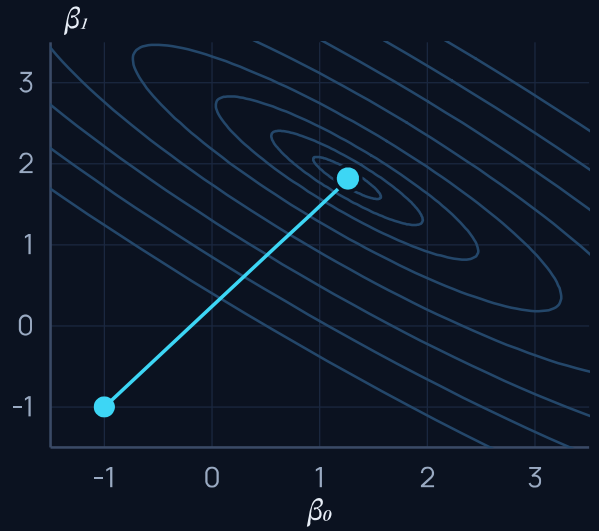
	Batch	Estocástico (SGD)	Mini-batch
Amostras por gradiente	todas (m)	1	b (ex.: 32)
Atualizações por época	1	m	m / b
Trajetória	suave	ruidosa	levemente ruidosa
Custo por atualização	alto	muito baixo	baixo
Vantagem	gradiente exato, estável	rápido para começar, cabe em memória	equilíbrio; usa vetorização e GPU
Limitação	lento em bases grandes	oscila perto do mínimo	mais um hiperparâmetro (b)
Uso típico	bases pequenas, esta aula	fluxos de dados, aprendizado online	redes neurais, padrão da indústria

LEGENDA m: número de amostras · b: tamanho do lote (batch size). As três usam a mesma regra $\beta \leftarrow \beta - \alpha \nabla J$; muda só quantas amostras entram no gradiente.

Três trajetórias até o mesmo mínimo

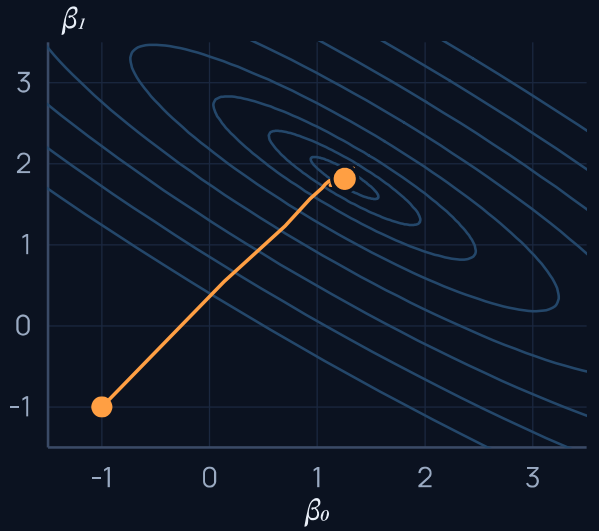
50 amostras sintéticas · ponto inicial (−1, −1) · α = 0,2 · 30 épocas para cada método.

BATCH



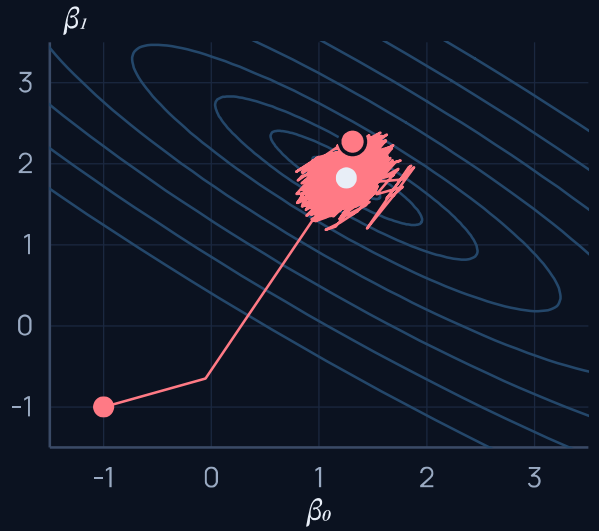
30 atualizações · final (1,26; 1,82)

MINI-BATCH · B = 10



150 atualizações · final (1,25; 1,82)

ESTOCÁSTICO



1.500 atualizações · final (1,31; 2,27)

LEITURA Elipses: curvas de nível de J, pontos com o mesmo custo. Ponto branco: mínimo pela equação normal. Ponto colorido: β ao fim de cada época.

Montando X, β e y em NumPy

$$\underset{(m,\ n+1)\ =\ (3,\ 2)}{X} \cdot \underset{(n+1,)\ =\ (2,)}{\beta} = \underset{(m,)\ =\ (3,)}{\hat{y}} \qquad \underset{(m,)\ =\ (3,)}{y}$$

$$X = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix} \qquad \beta = \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix} \qquad \hat{y} = \begin{pmatrix} \beta_0 + 1\beta_1 \\ \beta_0 + 2\beta_1 \\ \beta_0 + 3\beta_1 \end{pmatrix} \qquad y = \begin{pmatrix} 2 \\ 4 \\ 6 \end{pmatrix}$$

```
1  x = np.array([1.0, 2.0, 3.0])          # (3,)
2  y = np.array([2.0, 4.0, 6.0])          # (3,)
3  m = len(y)                             # 3 amostras
4  X = np.column_stack([np.ones(m), x]) # (3, 2)
5  beta = np.zeros(X.shape[1])            # (2,): [b0, b1]
```

4 A coluna de 1s multiplica β_0: é ela que cria o intercepto.

5 Começamos com todos os coeficientes em zero.

LEGENDA m: amostras (linhas de X) · n: características (colunas sem os 1s; aqui $n = 1$) · (3,) vetor com 3 elementos · (3, 2) matriz com 3 linhas e 2 colunas

Versão 1: o algoritmo com laço explícito

```python
import numpy as np

x = np.array([1.0, 2.0, 3.0])
y = np.array([2.0, 4.0, 6.0])
m = len(x)                          # m = 3

b0, b1 = 0.0, 0.0                   # β₀ e β₁ iniciais
alpha = 0.1                         # α

for t in range(1000):               # t: iteração
    y_hat = b0 + b1 * x             # ŷ = β₀ + β₁x
    erro = y_hat - y                # ŷ - y
    grad_b0 = erro.sum() / m            # ∂J/∂β₀
    grad_b1 = (erro * x).sum() / m      # ∂J/∂β₁
    b0 = b0 - alpha * grad_b0
    b1 = b1 - alpha * grad_b1
```

11 Previsões para todas as amostras.

12 Erro com sinal: positivo se o modelo prevê alto demais.

13 Erro médio: derivada de β_0.

14 Erro médio ponderado por x: derivada de β_1.

15–16 Passo contra a inclinação. Os dois gradientes já foram calculados com os valores antigos.

Versão 2: o algoritmo vetorizado

```
1  alpha, epocas, tol = 0.1, 1000, 1e-8
2  historico = []
3  for t in range(epocas):
4      y_hat = X @ beta
5      erro = y_hat - y
6      custo = (erro @ erro) / (2 * m)
7      historico.append(custo)
8      gradiente = (X.T @ erro) / m
9      beta = beta - alpha * gradiente
10     if t > 0 and abs(historico[-2] - historico[-1]) < tol:
11         break
```

1 α, máximo de épocas e ε.

2 Lista que guarda J de cada época.

3 t: índice da iteração. No batch, 1 iteração = 1 época.

4 $\hat{y} = X\beta$ `(m,)`

5 $\hat{y} - y$ `(m,)`

6 $J(\beta)$; erro @ erro = $\Sigma(\hat{y}_i - y_i)^2$ `escalar`

7 Registra o custo antes de atualizar: historico[0] é o custo inicial.

8 $\nabla J = X^T(\hat{y} - y) / m$ `(n+1,)`

9 $\beta \leftarrow \beta - \alpha \nabla J$: todos os coeficientes de uma vez.

10-11 Para se $|J(t-1) - J(t)| < \varepsilon$.

ANTES `X`, `y`, `m` e `beta` vêm da montagem de X. Com x = [1, 2, 3] e y = [2, 4, 6], a primeira passagem dá custo 9,3333 e beta = [0,4; 0,9333].

X^T faz as duas somas de uma vez

$$\underset{(2,\,3)}{X^T} \cdot \underset{(3,)}{\hat{y}-y} = \underset{(2,)}{\begin{bmatrix} -2-4-6 \\ 1(-2)+2(-4)+3(-6) \end{bmatrix}} = \begin{bmatrix} -12 \\ -28 \end{bmatrix} \div 3 = \underset{(2,)}{\begin{bmatrix} -4 \\ -9{,}33 \end{bmatrix}}$$

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix}$$

$$\nabla J$$

LINHA DE 1S

Soma os erros: $\Sigma(\hat{y}_i - y_i)$. Vira $\partial J/\partial \beta_0$.

LINHA DE X

Soma os erros ponderados: $\Sigma(\hat{y}_i - y_i)x_i$. Vira $\partial J/\partial \beta_1$.

OPERADOR @

Multiplicação de matrizes. Formatos: (a, b) @ (b,) → (a,).

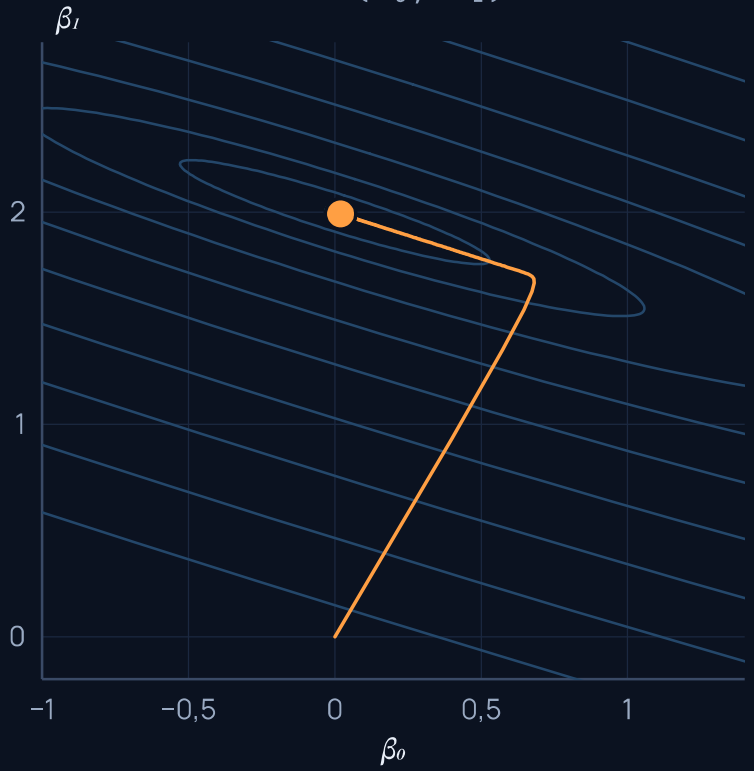
Dicionário de símbolos

x **entrada** característica de uma amostra `x`	y **valores reais** vetor do que queremos prever `y  (m,)`	$\hat{y}$ **previsões** saída do modelo, Xβ `y_hat`	β **coeficientes** parâmetros que o modelo aprende `beta  (n+1,)`
β_0 **intercepto** ŷ quando x = 0 `beta[0]`	β_1 **coef. angular** variação de ŷ por unidade de x `beta[1]`	X **matriz de características** 1ª coluna de 1s `X  (m, n+1)`	m **nº de amostras** linhas de X `m = len(y)`
n **nº de características** colunas de X sem os 1s `X.shape[1] - 1`	$J(\beta)$ **função de custo** erro quadrático médio ÷ 2 `custo`	α **taxa de aprendizado** tamanho do passo `alpha`	∂ **derivada parcial** inclinação em uma direção `gradiente[j]`
∇J **gradiente** vetor de derivadas parciais `gradiente`	t **iteração** contador do laço de treino `for t in range(epocas`	ε **tolerância** limite do critério de parada `tol`	Σ **somatório** soma sobre as amostras `.sum()  ou  @`

O ponto no mapa de custo e a reta nos dados

α = 0,1 · β inicial = (0, 0) · dados x = [1, 2, 3], y = [2, 4, 6]

CURVAS DE NÍVEL DE J(B₀, B₁)



RETA DA ITERAÇÃO ATUAL



CUSTO POR ITERAÇÃO (LOG)



t = 300 β₀ = 0,020 β₁ = 1,991 J = 0,0000

Calculando J em uma grade para desenhar o mapa

```
1  trajetoria = [beta.copy()]                # antes do laço
2  trajetoria.append(beta.copy())            # no laço, após atualizar
3  traj = np.array(trajetoria)               # (épocas + 1, 2)
4
5  B0, B1 = np.meshgrid(np.linspace(-1, 1.5, 200),
6                       np.linspace(-0.5, 3, 200))   # (200, 200)
7  E = B0[..., None] + B1[..., None] * x - y       # (200, 200, m)
8  J_grade = (E ** 2).sum(axis=-1) / (2 * m)       # (200, 200)
9
10 plt.contour(B0, B1, J_grade, levels=20)
11 plt.plot(traj[:, 0], traj[:, 1], 'o-')
```

meshgrid

Cria todas as combinações de β_0 e β_1: cada posição da grade é um palpite.

[..., None]

Acrescenta um eixo: (200, 200) vira (200, 200, 1) e combina com x (m,).

Linha 8

A fórmula de J aplicada a todos os palpites: soma no eixo das amostras, divide por 2m.

3D

ax.plot_surface(B0, B1, J_grade) desenha a mesma grade como superfície.

Animando a trajetória com FuncAnimation

```python
from matplotlib.animation import FuncAnimation

fig, ax = plt.subplots()
ax.contour(B0, B1, J_grade, levels=20)
caminho, = ax.plot([], [], 'o-', color='C1')

def atualizar(k):                          # k: quadro = iteração
    caminho.set_data(traj[:k + 1, 0], traj[:k + 1, 1])
    J = ((X @ traj[k] - y) ** 2).sum() / (2 * m)
    ax.set_title(f't = {k}   J = {J:.4f}')
    return caminho,

anim = FuncAnimation(fig, atualizar, frames=len(traj),
                     interval=100)
anim.save('descida.gif', writer='pillow', fps=10)
```

`caminho, =`

ax.plot devolve uma lista; a vírgula pega o único elemento: a linha vazia.

`atualizar(k)`

Chamada uma vez por quadro. Redesenha β das iterações 0 até k.

`set_data`

Troca os pontos da linha sem criar um gráfico novo.

`frames · interval`

Quantos quadros e quantos milissegundos entre eles.

NO NOTEBOOK `HTML(anim.to_jshtml())` mostra a animação na célula. Cada quadro é uma iteração t: o ponto anda sobre o mapa enquanto J diminui.

Exercício guiado em oito etapas

Notebook construído célula por célula. Confira a saída esperada antes de avançar.

01	**Gerar dados** x, y com rng.uniform e ruído normal	`x (100,) · y (100,)`	
02	**Montar X** coluna de 1s + x	`X (100, 2)`	
03	**Inicializar** beta = zeros, α, épocas, ε	`beta (2,)`	
04	**Função custo** J(β) = erro @ erro / 2m	`escalar`	
05	**Função gradiente** $X^T(X\beta - y) / m$	`(2,)`	
06	**Laço de treino** atualizar, registrar, parar	`beta, historico`	
07	**Prever e plotar** reta final e custo por época	`2 gráficos`	
08	**Animar** curvas de nível + FuncAnimation	`animação`	

Erros comuns: configuração e dimensões

Taxa de aprendizado alta
custo cresce, vira inf ou nan

```
ERRO alpha = 5.0
```

```
CERTO alpha = 0.1  # e observe o custo
```

Épocas insuficientes
β longe do esperado; custo ainda caindo

```
ERRO epocas = 10
```

```
CERTO epocas = 5000; tol = 1e-8
```

Esquecer o intercepto
reta forçada a passar pela origem

```
ERRO X = x.reshape(-1, 1)
```

```
CERTO X = np.column_stack([np.ones(m), x])
```

Dimensões incorretas
y (m, 1) − ŷ (m,) vira matriz (m, m)

```
ERRO y = y.reshape(-1, 1)
```

```
CERTO y = y.ravel()
      assert erro.shape == (m,)
```

Não registrar o histórico
impossível saber se aprendeu

```
ERRO for t in range(epocas): ...
```

```
CERTO historico.append(custo)
```

Erros comuns: sinais e escala

Sinal trocado
custo cresce mesmo com α pequeno

```
ERRO beta = beta + alpha * gradiente
```

```
CERTO beta = beta - alpha * gradiente
```

Custo no lugar do gradiente
todos os β mudam igual, sem direção

```
ERRO beta = beta - alpha * custo
```

```
CERTO beta = beta - alpha * gradiente
```

Erro invertido
gradiente aponta para cima

```
ERRO erro = y - y_hat
```

```
CERTO erro = y_hat - y
```

Escalas muito diferentes
área ≈ 200, quartos ≈ 3: α não serve para os dois

```
ERRO X = np.column_stack(
       [np.ones(m), area, quartos])
```

```
CERTO Z = (F - F.mean(0)) / F.std(0)
      X = np.column_stack([np.ones(m), Z])
```

Quiz de saída

Se você responde as oito, consegue implementar e explicar o gradiente descendente.

01 O que é β e por que existem β_0 e β_1?

02 O que α representa e o que acontece se for muito alto?

03 Por que subtraímos o gradiente?

04 O que acontece quando o gradiente se aproxima de zero?

05 O que significa uma curva de custo decrescente?

06 Como identificar divergência?

07 Qual é a diferença entre batch e SGD?

08 O que a linha X.T @ erro / m calcula?

Desafio de casa: a classe GradienteDescendente

```python
class GradienteDescendente:
    def __init__(self, alpha=0.1, epocas=1000, tol=1e-8):
        ...   # guarda os hiperparâmetros
    def fit(self, X, y):
        ...   # adiciona 1s, treina beta_, salva historico_
    def predict(self, X):
        ...   # retorna as previsões ŷ
    def score(self, X, y):
        ...   # retorna o R²
```

R² · COEFICIENTE DE DETERMINAÇÃO

$$R^2 = 1 - \frac{\Sigma (y_i - \hat{y}_i)^2}{\Sigma (y_i - \bar{y})^2}$$

ȳ é a média de y. R² = 1: previsão perfeita. R² = 0: igual a prever sempre a média.

CRITÉRIOS DE ACEITE

01 O código executa sem erros.

02 Os coeficientes se aproximam dos valores esperados.

03 O custo diminui com um treino bem configurado.

04 predict gera previsões corretas e score devolve o R².

05 A animação mostra a trajetória de β sobre as curvas de nível.

06 Você explica β, α, gradiente e custo com suas palavras.

Leitura: documentação oficial

NUMPY

numpy.matmul · @
Multiplicação de matrizes e regras de formato.

numpy.ndarray.T
Transposta de um array.

numpy.column_stack
Montar X com a coluna de 1s.

Broadcasting
Por que (m, 1) − (m,) vira (m, m).

numpy.linalg.pinv
Pseudoinversa, para comparar resultados.

MATPLOTLIB E GERAÇÃO DE DADOS

Axes.contour
Curvas de nível da função de custo.

Axes3D.plot_surface
Superfície tridimensional do custo.

animation.FuncAnimation
Animação da trajetória de β.

numpy.meshgrid
Grade de (β_0, β_1) para avaliar J.

numpy.random.Generator
Dados sintéticos reprodutíveis.